PhoenixGuard: End-to-End System Blueprint

1. Introduction

PhoenixGuard is a hybrid chart intelligence system for financial signal review, memory-augmented reasoning, and multi-module decision support. This document provides a comprehensive index and architectural overview, explaining how each component is built, how they connect, and their purpose in the end-to-end pipeline.

2. High-Level Architecture Mind Map

See next page for diagram.

PhoenixGuard Architecture Mind Map

User Input (Image/File) → Preprocess (preprocess.py) → CV Model Detection (cv_module.py) → Chart-State Extraction → Memory Retrieval & Context Injection (memory_ingest.py) → Regression Forecasting (regression_module.py) → Style Personalization (personalization.py) → RL Policy Inference (rl_module.py) → Feature Fusion (main.py) → Gate Layer (skill_gates.py) → Ensemble Decision (ensemble.py) → Final Action & Outputs → Online Adaptation Loop (feeds back to Memory Retrieval)

3. System Overview Table

Layer	Main Components & Files	Purpose
Input	main.py (run_inference)	Accepts user chart/image input
Preprocessing	preprocess.py	Loads and normalizes input for models
CV Model	cv_module.py	Detects chart features using vision models
Chart-State Extraction	main.py, cv_module.py	Extracts structured chart state from detections
Memory Retrieval	memory_ingest.py, main.py	Retrieves similar past cases for context and recal
Regression Forecasting	regression_module.py	Predicts future chart values/quantiles
Personalization	personalization.py	Adapts style and behavior to user profile
RL Policy Inference	rl_module.py	Makes policy decisions using reinforcement learn
Feature Fusion	main.py	Combines all features into a single vector
Gate Layer	skill_gates.py	Applies 12 formal gates for signal validation and
Ensemble Decision	ensemble.py	Fuses all signals, applies consensus rules for fina
Final Action & Outputs	main.py, ensemble.py	Generates actionable output (BUY/SELL/HOLD
Online Adaptation Loop	main.py, personalization.py	Continuously adapts using feedback and memory

4. End-to-End Flow Explanation

Step 1: Input User provides a chart image or file. Entry point: run_inference in main.py.

Step 2: Preprocessing Image is loaded and normalized for model compatibility (preprocess.py).

Step 3: CV Model Detection Vision models (cv_module.py) detect chart features and patterns.

Step 4: Chart-State Extraction Detected features are structured into a chart-state payload.

Step 5: Memory Retrieval & Context Injection System retrieves similar past cases from the memory bank (memory_ingest.py). Few-shot context is built for reasoning.

Step 6: Regression Forecasting Predicts future chart values using regression models (regression_module.py).

Step 7: Personalization Updates user style and adapts outputs (personalization.py).

Step 8: RL Policy Inference RL engine makes policy decisions based on fused features (rl_module.py).

Step 9: Feature Fusion All features are combined into a single vector (main.py).

Step 10: Gate Layer 12 formal gates validate and refine the signal (skill_gates.py).

Step 11: Ensemble Decision All signals are fused and consensus rules applied (ensemble.py).

Step 12: Final Action & Outputs System outputs BUY/SELL/HOLD, position sizing, and visual explanations.

Step 13: Online Adaptation Loop Feedback and memory updates continuously improve the system.

5. Component Purposes

main.py: Orchestrates the pipeline, manages input/output, and feature fusion.
preprocess.py: Handles all input normalization and preprocessing.
cv_module.py: Runs computer vision models for chart feature detection.
memory_ingest.py: Manages memory bank retrieval and context injection.
regression_module.py: Provides regression-based forecasting.
personalization.py: Adapts system behavior to user preferences.
rl_module.py: Implements reinforcement learning policy inference.
skill_gates.py: Applies formal gates for signal validation.
ensemble.py: Fuses all signals and applies consensus logic.

6. Conclusion

PhoenixGuard is a modular, extensible, and adaptive system for advanced chart intelligence and decision support, with each component playing a critical role in the end-to-end workflow.